

Потоки, процессы, параллельная работа.

В современных многопроцессорных и многоядерных вычислительных системах все вычисления выполняются в параллельном режиме на разных процессорах. Управление распределением вычислительных ресурсов выполняется операционной системой в первую очередь для своих задач (обеспечение эффективной работы системы в целом). Этими ресурсами являются собственно вычислительные устройства (ядра, процессоры), память, а также внешние устройства ввода/вывода и обмена данными.

Для вычислительных систем вводится понятие процесса как отдельной единицы, отвечающей за решение определенной задачи в ходе ее выполнения на компьютере. Для выполнения процесса ему необходимы ресурсы — память, процессор и, возможно, другие устройства для ввода/вывода и т.п. Операционная система ведет регистрацию запущенных процессов, предоставляет им необходимые ресурсы и тем самым обеспечивает выполнение этих процессов. Как правило, в системе одновременно присутствует множество процессов и система распределяет ресурсы между ними в соответствии с некоторой логикой, обеспечивающей эффективное решение поставленных перед процессами задач. С точки зрения распределения ресурсов процессора все просто. Процессор — это отдельное физическое устройство, которое может выполнять заданные команды, и системе достаточно давать указания каждому доступному процессору выполнять тот или иной код в указанные промежутки времени (по той или иной логике организации очередей). С памятью дело обстоит по-другому. Многозадачность обслуживается системой виртуальной памяти, когда каждый процесс получает в свое распоряжение некоторый диапазон “виртуальных адресов”, а отображение этих адресов на схемы реальной физической памяти выполняется специальными системными и аппаратными механизмами. Кроме этого, для выполнения задачи требуется также область памяти, реализующая стек для организации эффективного вызова функций и возврата результатов этих функций. Вот эти три компонента — процессор, память, стек и являются тремя основными ресурсами, обеспечивающими работу процесса в системе. То есть, создавая новый процесс, система должна позаботиться о выделении памяти, стека и процессорного времени для каждого конкретного процесса. Среди этих трех компонент управление памятью является самой затратной процедурой поскольку требует поддержки таблиц отображения адресов для управления виртуальной памятью. В связи с этим, в системах существует два механизма поддержки параллелизма — процессы (process) и потоки/нити (thread).

Процессы являются полностью независимыми друг от друга в плане трех основных компонент ресурсов. Однако на уровне системы между процессами поддерживаются “родительские” отношения, т.е. каждый новый процесс возникает не сам по себе, а является результатом выполнения определенных действий в другом процессе, который тем самым его “порождает”.

Потоки/нити также порождаются выполнением некоторых команд в текущем процессе. Однако в этом случае они сохраняют доступ к памяти исходного процесса, но получают в свое распоряжение отдельные стеки и возможность выполняться на других процессорах в системе. Родительских отношений между порожденными потоками уже нет, можно говорить, что они в каком-то смысле становятся полностью равноправными.

На данном этапе мы рассмотрим способы работы с потоками в рамках простейших прикладных программ. Далее уже не будет подробных разъяснений и обсуждений, а лишь упоминание различных функций и простейшие примеры. Предполагается, что читатель способен самостоятельно найти требуемую дополнительную информацию в разнообразных справочных источниках.

Пока рассматриваем простейший C интерфейс и потихоньку добавляем C++.

Заголовочные файлы

```
#include <pthread.h>           // собственно потоки
#include <unistd.h>             // некоторые системные функции для удобства
```

Типы данных

```
pthread_t           // для идентификации потока
pthread_attr_t      // атрибуты потока
```

Функции работы с потоками (про параметры позже)

```
pthread_create      // создать
pthread_join        // ждать окончания
pthread_cancel      // отменить
```

```
pthread_detach    // отсоединить
pthread_exit      // завершить
```

Примеры.

timer.cpp — простенькая программа для демонстрации асинхронного поведения потоков и работы базовых функций.

parint.cpp — параллельное интегрирование — демонстрация содержательного распараллеливания вычислительной работы и сокращения времени выполнения программы.

Задачи для самостоятельного решения.

Реализовать функции для параллельного расчета простейших вычислительных задач. Функция должна получать количество потоков через параметр `a` также все другие параметры и данные, необходимые для расчета. Замер времени работы проводится вне этих функций. Требуется протестировать реализации, провести сравнение времени вычислений для одиночного потока и для различного количества потоков. Нужно также выяснить как ускорение работы при использовании нескольких потоков зависит от количества потоков и объема (размерности) входных данных.

Примеры задач:

1. скалярное произведение векторов;
2. умножение матрицы на вектор;
3. произведение матриц прямоугольных матриц.

Задачи на поток-таймер (синхронизация).

4. Основной поток занят серьезными делами. Таймер отсчитывает время в глобальную переменную. По мере завершения этапов этих серьезных дел основной поток проверяет сколько времени прошло и печатает время и процент выполненной работы на этот момент. `u`
5. Основной поток занят серьезными делами и по мере их выполнения записывает в глобальную переменную сколько процентов работы завершено. Отдельный поток отсчитывает время и проверяет эту глобальную переменную. Когда значение переменной изменяется, этот поток печатает процент выполненной работы и текущее время.
6. Основной поток занят серьезными делами и периодически должен печатать результаты. Результаты появляются не совсем регулярно, но печатать их надо обязательно в моменты времени кратные определенному промежутку времени (например, один раз в секунду). Если к очередному моменту результатов еще нет (не успели посчитать), то надо печатать сообщение "no results а если получены раньше, то надо подождать с печатью до наступления очередного момента.
7. Программа запускает 3 потока, которые занимаются продолжительной работой, а сама основная программа работает как таймер, отсчитывающий время. Как только конкретный запущенный поток завершает свою работу, основная программа должна напечатать его номер (идентификатор) и затраченное им время. Таким образом мы получим последовательный список в каком порядке потоки закончили работу.
8. Программа решает серию вычислительных задач, последовательно запуская для каждой задачи отдельный поток (именно не все сразу, а один за другим последовательно). На решение каждой задачи отводится определенное время (например, одинаковое — 1 секунда, но можно и разные). Если поток, решающий задачу, не укладывается в отведенное ему время, то решение отменяется, поток сбрасывается и при этом печатается соответствующее сообщение. Если же укладывается, то печатается время, затраченное на решение.

Для первых двух задач серьезным делом может быть, например, подсчет интеграла как в нашем примере (работает несколько секунд). Для других задач — подсчет серии интегралов от разных функций с разными параметрами (чтобы время подсчета оказалось разным).

Отчетность по задачам устная с демонстрацией того, что и как сделано, объяснением почему именно так и итоговыми выводами по результатам тестов (удалось ли сделать, какие выводы по ускорению работы, какие способы синхронизации применялись и почему и т.п.).